

BabyShopHub

System Design Document

Aptech eProject Submission

Version 1.0

Table of Contents

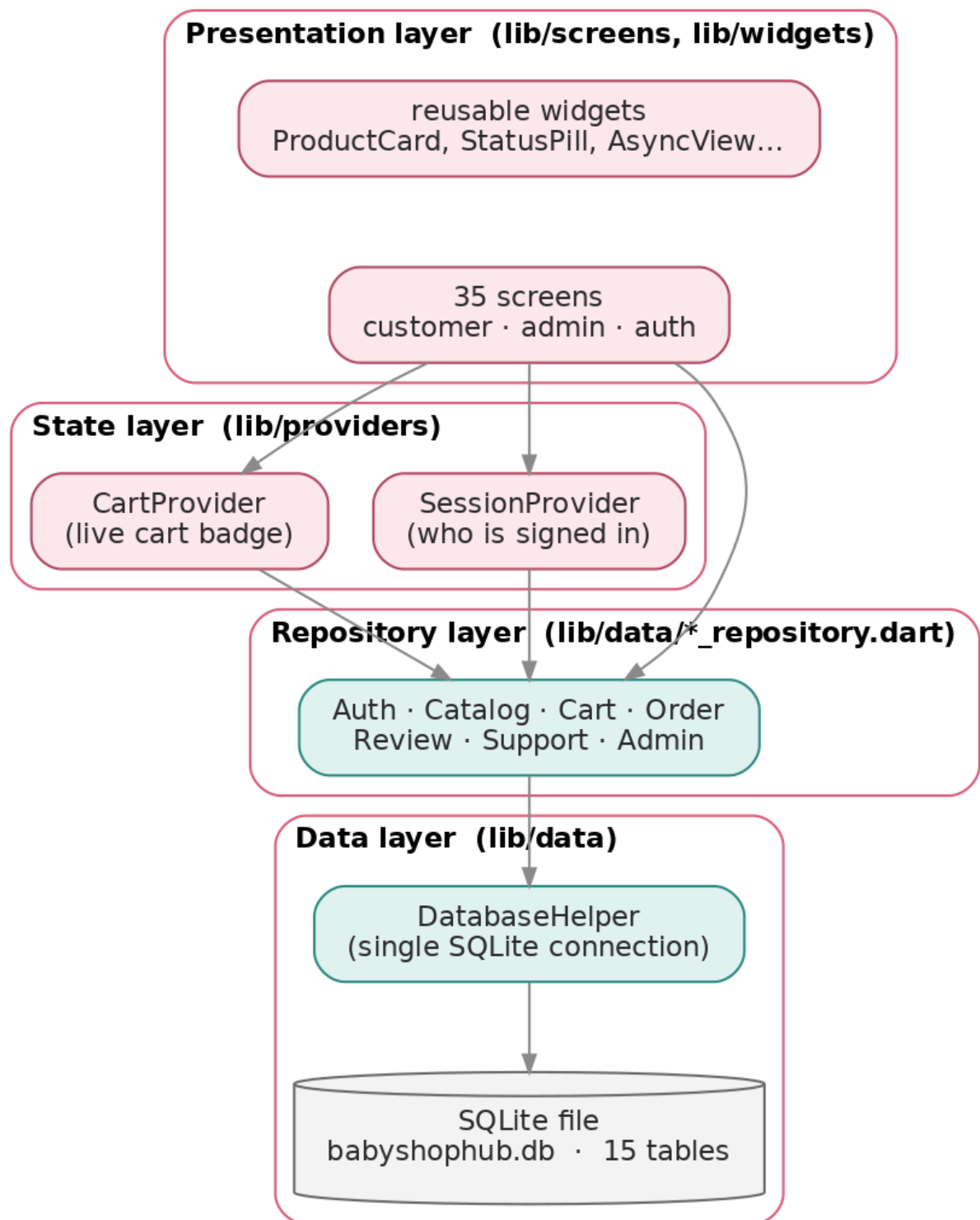
1. Purpose of this document.....	2
2. Architecture overview.....	3
2.1 Why this structure.....	5
3. Technology choices.....	6
4. Data model.....	7
4.1 Tables at a glance.....	7
4.2 Two deliberate modelling decisions.....	9
4.3 Referential integrity.....	9
5. State management.....	10
5.1 Routing at the root.....	10
6. Security design.....	11
6.1 Password storage.....	11
6.2 Access control.....	11
6.3 What the dummy payment stores.....	11
7. Order lifecycle (state machine).....	12
8. Support ticket lifecycle.....	13
9. Screen map.....	14
10. How the design meets the non-functional requirements.....	15

1. Purpose of this document

This document explains **how** BabyShopHub is built, where the SRS explains **what** it does. It describes the architecture, the data model, the way the app holds state, the security design, and the two state machines that govern orders and support tickets. It is written so that a developer can understand the shape of the system before opening the code, and so that a reviewer can see that the design meets the non-functional requirements.

2. Architecture overview

BabyShopHub uses a layered architecture. Each layer only talks to the layer directly below it, which keeps responsibilities separate and makes the system easy to reason about and to extend.



Architecture layers

Presentation layer (lib/screens, lib/widgets). The 35 screens and the reusable widgets. Screens never touch the database directly; they call a repository or read a provider.

Common UI — product cards, status pills, loading/empty states — is factored into shared widgets so every screen looks and behaves consistently.

State layer (lib/providers). Two ChangeNotifier providers hold the small amount of state that is shared across screens: SessionProvider (who is signed in, and therefore what the app is allowed to show) and CartProvider (the live cart, so the cart badge and totals update the instant an item is added anywhere).

Repository layer (lib/data/*_repository.dart). Every data operation the app can perform is a method on a repository — AuthRepository.login, OrderRepository.placeOrder, AdminRepository.setUserActive, and so on. Repositories translate between database rows and the typed model objects the UI uses. Write methods return a String?: null on success, or a human-readable message on failure, which is what lets every screen show a clear error instead of crashing.

Data layer (lib/data). A single DatabaseHelper owns the one SQLite connection and the schema. It is the only class that knows what the storage looks like. Because everything is local, this layer never makes a network call.

2.1 Why this structure

This separation is what delivers several of the non-functional requirements at once. Responsiveness (NFR-1) follows from all data being local and read through indexed queries. Maintainability (NFR-7) follows from each concern living in one layer, with single sources of truth for pricing, statuses, and theme. Scalability (NFR-6) follows from being able to add a screen or a repository method without disturbing the others.

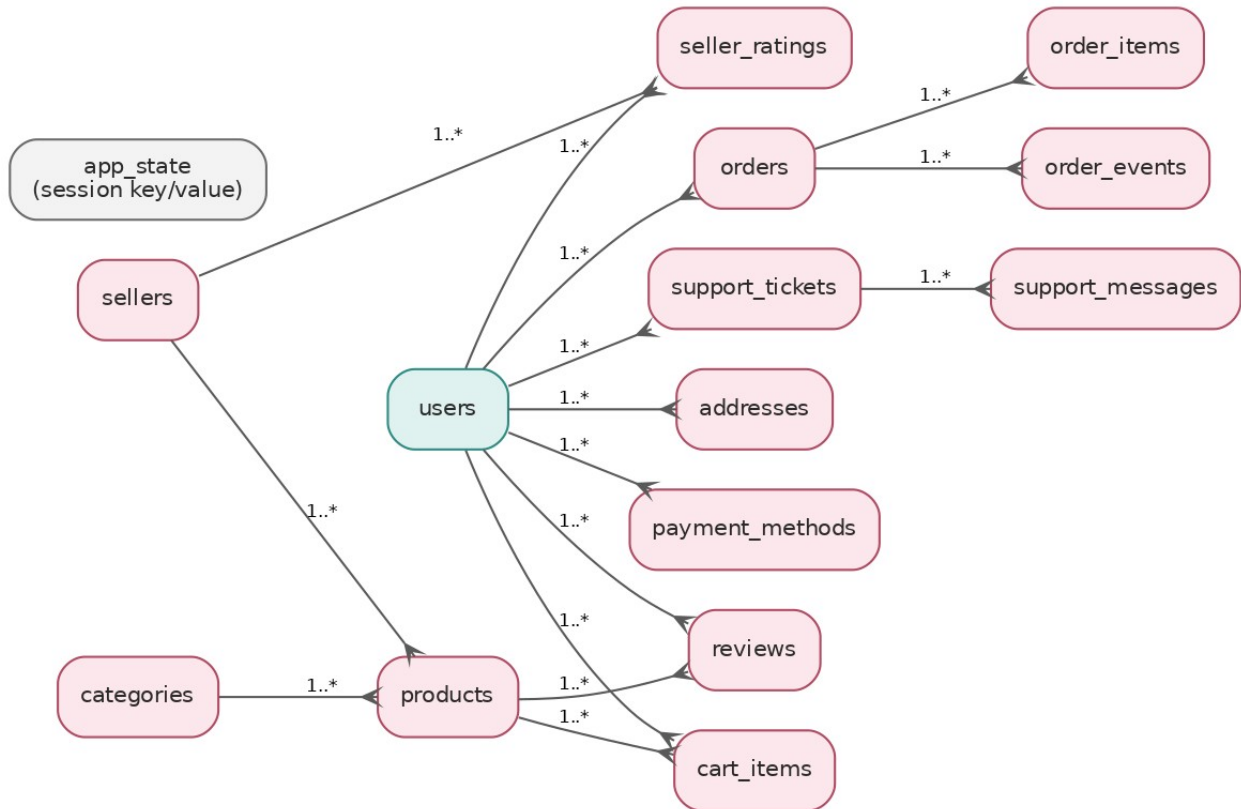
3. Technology choices

Concern	Choice	Why
UI framework	Flutter (Material 3)	One codebase for Android, iOS, and desktop; a mature widget set.
Language	Dart	Flutter's language; sound null safety catches whole classes of error at compile time.
Database	SQLite via sqflite	A real relational store, embedded in the app, no server needed.
Desktop database	sqflite_common_ffi + sqlite3_flutter_libs	The same code runs on Windows/Linux for demonstration.
State management	provider	Lightweight, official, and enough for this app's shared state.
Password hashing	crypto (SHA-256)	Standard, well-tested hashing for salted password digests.
Path handling	path	Builds the database file path correctly on every platform.

The dependency list is deliberately short. Every package is widely used and actively maintained, which keeps the project easy to build and to trust.

4. Data model

All persistent data lives in one SQLite database, `babyshophub.db`, created on first launch. The schema has **15 tables**. Foreign keys are enforced (`PRAGMA foreign_keys = ON`), and every column the app filters or joins by is indexed (15 indexes in total).



Entity relationships

4.1 Tables at a glance

Table	Holds	Key relationships
users	Accounts: name, email, salted password hash, role, active flag	Parent of most tables
app_state	A key/value store; holds the signed-in user id for session persistence	—
addresses	Saved delivery addresses	belongs to users (cascade)
payment_meth	Saved cards —	belongs to users (cascade)

ods	brand and last four digits only	
categories	Product categories with an icon and sort order	parent of products
sellers	Sellers, with recomputed rating totals	parent of products
products	Catalogue: price, stock, image, rating totals, active flag	references categories, sellers
cart_items	The live cart; unique per (user, product)	references users, products
orders	Placed orders with a copied shipping address and price breakdown	belongs to users
order_items	Line items, with product details copied at checkout	belongs to orders (cascade)
order_events	The tracking timeline — one row per status reached	belongs to orders (cascade)
reviews	Product star ratings and comments; unique per (product, user)	references products, users
seller_ratings	Seller star ratings; unique per (seller, user)	references sellers, users
support_tickets	Support conversations with a status	belongs to users
support_messages	Messages within a ticket, tagged by sender role	belongs to support_tickets (cascade)

4.2 Two deliberate modelling decisions

Order data is copied, not referenced. When an order is placed, the product name, brand, image, and unit price are copied into `order_items`, and the delivery address is copied onto the `orders` row. `order_items` intentionally carries **no** foreign key to `products`. This means an administrator can later edit the price of a product or delete a discontinued one without ever changing or corrupting a past receipt. A historical order is a complete, immutable record of what was bought and where it was sent.

Rating totals are recomputed, never incremented. Each product and seller stores a `rating_sum` and `rating_count`. Rather than adjusting these by hand on each new review, the app recomputes them directly from the rating rows after any change (add, edit, delete, or hide). This guarantees the displayed average can never drift away from the reviews a shopper can actually read — a hidden review, for instance, is excluded automatically.

4.3 Referential integrity

Foreign keys use two different delete rules by design:

- **CASCADE** on data that belongs to a parent and has no meaning without it: a user's addresses, cards, cart, reviews, and support tickets; an order's items and events; a ticket's messages. Removing the parent cleanly removes these.
- **NO ACTION** on references that must not trigger silent deletion: `products.category_id`, `products.seller_id`, and `orders.user_id`. Deleting a category must not wipe its products, so the admin screens check for dependants first and refuse (or offer to deactivate) instead.

5. State management

The app keeps shared state deliberately small. Most screens simply load what they need from a repository when they open. Only two pieces of state are truly cross-cutting, and each has its own provider:

SessionProvider holds the currently signed-in `AppUser`, or null. It is restored on launch by reading the saved user id from the `app_state` table, so a returning user is taken straight to the right home screen. Signing in or out updates this provider, and the root of the app listens to it to decide which screen to show.

CartProvider holds the current user's cart. It is bound to the signed-in user, exposes the item count and totals, and notifies listeners whenever the cart changes — so the cart badge, the cart screen, and the checkout button all stay in step without passing data between screens manually.

5.1 Routing at the root

A small gate at the root of the app (`_RootGate` in `main.dart`) watches `SessionProvider` and chooses the screen:

- still restoring the saved session → the splash screen;
- not signed in → the login screen;
- signed in as an administrator → the admin shell;
- signed in as a customer → the customer shell.

This single decision point is where the security rule “only registered users can reach customer features, only administrators can reach the admin panel” is enforced structurally: an unauthenticated or customer session simply never has the admin shell built for it.

6. Security design

Security is a stated requirement (NFR-5), and the design addresses it on two fronts: credentials and access.

6.1 Password storage

Passwords are never stored, logged, or compared in plain text. When an account is created, the app generates a random **salt** unique to that account, and stores the **SHA-256 hash** of the salt combined with the password. At login, the app hashes the entered password with the stored salt and compares digests. Two users with the same password therefore have completely different stored hashes, and the stored value cannot be reversed to reveal the password.

6.2 Access control

Access is controlled by the role on the account (customer or admin) and the `is_active` flag:

- The root gate (Section 5.1) routes each role to its own area and never builds the admin panel for a customer session.
- Administrative operations live behind the admin shell, which is only reachable by an admin session.
- A suspended account (`is_active = 0`) is refused at login.
- The system refuses to suspend or demote the **last active administrator**, so the shop can never lock itself out of management.

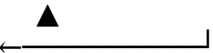
6.3 What the dummy payment stores

The checkout validates card input for format only and processes no real payment. Only the card brand and the last four digits are ever saved; the full number is used to derive those and then discarded, and no security code is ever collected. This keeps the demonstration realistic without holding sensitive data.

7. Order lifecycle (state machine)

An order moves through a deliberately linear flow. The customer's tracking screen draws this as a timeline; the admin order screen offers only the **permitted next transitions** from the current state.

Pending → Confirmed → Packed → Shipped → Out for delivery → Delivered

Cancelled ←  (only reachable before Shipped)

The rules, enforced in `OrderStatus.nextOptions`:

- From any state on the happy path, the only forward move is to the **next** state in the sequence — statuses cannot be skipped.
- **Cancelled** is offered only while the order has not yet reached **Shipped**; once a parcel is with the courier it can no longer be cancelled in-app.
- **Delivered** and **Cancelled** are final — an order in either state offers no further transitions.

Every transition writes a row into `order_events` with a timestamp and a friendly note, which is exactly what the customer sees as their tracking history.

8. Support ticket lifecycle

Support tickets have four states: **Open**, **In progress**, **Resolved**, and **Closed**. A new ticket starts **Open**. When the administrator replies, the ticket moves to **In progress**. The administrator can mark it **Resolved** or **Closed** when the issue is handled, and can **reopen** a closed ticket back to **Open** if the customer comes back. Each message is tagged with its sender role (customer or support), which is what lets both sides of the conversation be laid out correctly on each person's screen.

9. Screen map

The 35 screens divide into four groups.

Entry (3): splash (restores the session), login, register.

Customer storefront (19): the customer shell (bottom navigation) hosting home, categories, product list (with search), product detail, cart, checkout, payment form, order confirmation, orders list, order detail (tracking), write review, account, edit profile, address list and form, payment methods and form, and the support list, support detail, new-ticket, and FAQ screens.

Admin panel (11): the admin shell (bottom navigation + a “More” tab) hosting the dashboard, orders and order detail, products and product form, categories, users, reviews, and support and support detail.

Shared: reusable widgets used across all of the above.

Each screen maps to one or more requirements in the SRS traceability matrix (SRS Section 5).

10. How the design meets the non-functional requirements

Requirement	How the design meets it
NFR-1 Responsiveness	All data is local; queries hit indexed columns; no network wait.
NFR-2 Usability	One theme, shared widgets, empty-state guidance, confirmation dialogs.
NFR-3 Accessibility	Legible type, adequate contrast and tap targets, labelled controls.
NFR-4 Error handling	Repository writes return a message on failure; input is validated first.
NFR-5 Security	Salted SHA-256 hashes; role-based routing; suspended-account and last-admin guards.
NFR-6 Scalability	Normalised, indexed schema; layered code that extends without churn.
NFR-7 Maintainability	Single sources of truth for pricing, statuses, and theme.
NFR-8 Portability	Same code on mobile and desktop; the data layer adapts per platform.

The Developer Guide expands Sections 2–5 into a working orientation of the codebase; the Test Plan verifies the behaviours designed here.